

DOSSIER CLASSIFIÉ — NIVEAU 5 — NE PAS DIVULGUER

OPÉRATION : BACKEND SECRET

*Bâtir l'API ultra-sécurisée de l'Agence S.P.I.O.N.***Laboratoire BONUS** • Service Web (25604)Durée : **3 heures** • Travail **en groupe** (2 à 3 agents)

Récompense : ce laboratoire bonus vaut **+5 %** appliqués à **l'un de vos 3 laboratoires** du cours (celui de votre choix, idéalement votre moins bon résultat).

Transmission entrante du Quartier Général... L'Agence **S.P.I.O.N.** (Service Privé d'Interception, d'Opérations et de Neutralisation) a un problème : ses dossiers d'agents et ses missions sont encore gérés sur des fiches cartonnées qu'un stagiaire a failli laisser dans un taxi. **Inacceptable.** Votre cellule est chargée de construire un **backend sécurisé** : recruter des agents sous couverture, gérer des missions classifiées, et garantir que *seules les bonnes personnes* accèdent aux *bonnes informations*. Bonne chance. Ce message s'autodétruit... jamais, parce qu'on n'a pas eu le budget.

Briefing : votre cible technologique

Votre backend doit obligatoirement combiner les 5 technologies vues en cours :

- **Express** — l'API REST (routes, routers, middlewares).
- **Prisma + Neon** — la base de données PostgreSQL (modèles, enums, relations, migrations).
- **Axios** — consommer une **API publique externe** pour enrichir vos données.
- **JWT + bcrypt** — l'authentification et l'autorisation (mots de passe hachés, tokens, rôles).

Règle d'engagement : cet énoncé ne contient **aucun code**. À vous de concevoir et d'implémenter. Un **aide-mémoire** (commandes et concepts) est fourni en **annexe** — rien de plus.

1. Mission 0 — Constituer la cellule

[5 pts]

Ordres

- Formez votre cellule (2 à 3 agents). Désignez un *dépôt Git* commun.
- Initialisez un projet Node + TypeScript et installez vos dépendances.
- Connectez une base **Neon** et vérifiez que Prisma la voit.
- Mettez en place vos variables secrètes (chaîne de connexion, secret de signature) **sans jamais les divulguer** dans le dépôt.

Répartissez le travail : pendant que l'un prépare la base de données, un autre peut amorcer le serveur Express. Commitez tôt, commitez souvent.

2. Mission 1 — Le Grand Fichier (modélisation)

[20 pts]

Concevez le schéma de la base. **Vous** décidez des noms exacts, mais votre modélisation doit contenir au minimum :

À concevoir

- Une entité **Agent** : possède au moins un identifiant de connexion unique, un mot de passe (qui sera *haché*, jamais en clair), un **rôle** et une **habilitation**.
- Une entité **Mission** : un titre, une description, un **niveau de confidentialité**, un **statut**, et une récompense.
- Une **relation** entre les deux : une mission est *confiée* à un agent (et un agent peut avoir plusieurs missions).
- Au moins **trois enums** bien pensés. Suggestions : le rôle (AGENT, CHEF), l'habilitation (CONFIDENTIEL, SECRET, TRES_SECRET), le statut d'une mission (DISPONIBLE, EN_COURS, REUSSIE, ECHOUÉE).

Pensez aux champs optionnels, aux valeurs par défaut et aux contraintes d'unicité. Un bon schéma rend les missions suivantes beaucoup plus simples. Appliquez votre migration avant de continuer.

3. Mission 2 — Recrutement sous couverture (Axios)

[20 pts]

Tout nouvel agent a besoin d'une **identité de couverture** crédible. Plutôt que de l'inventer, votre système ira la chercher auprès d'une **API publique** d'identités fictives.

Objectif opérationnel

- Créez une route qui, lors du recrutement d'un agent, **interroge une API publique** (via Axios) pour obtenir une fausse identité (nom de couverture, nationalité, photo...).
- **Transformez** la réponse (gardez seulement les champs utiles) et **enregistrez** l'agent dans votre base Neon.
- Une route permet de **lister** les agents recrutés ; une autre d'en **consulter** un seul par son identifiant.
- Gérez proprement le cas où l'API externe ne répond pas (erreur réseau ou statut d'erreur).

Une API publique gratuite et sans clé est suggérée en **Annexe** ; vous pouvez aussi en choisir une autre, du moment qu'elle ne demande pas d'authentification. Centraliser votre client Axios (une seule instance configurée) vous fera gagner du temps.

4. Mission 3 — Le contrôle d'accès (JWT)

[25 pts]

Une agence secrète sans contrôle d'accès, c'est une passoire. Mettez en place l'authentification.

Objectif opérationnel

- Une route d'**inscription** : crée un compte agent avec un mot de passe **haché** (jamais stocké en clair, jamais renvoyé dans la réponse).
- Une route de **connexion** : vérifie les identifiants et, si tout est bon, remet un **token signé** (avec une **expiration**).
- Un **middleware** qui protège les routes sensibles : il lit le token envoyé par le client, le **vérifie**, et rejette les intrus.
- Le token ne doit contenir **aucune information sensible** (rappel : son contenu est *lisible* par tous).

Contre-espionnage. Votre message d'erreur de connexion doit rester **générique** : ne révélez pas si c'est l'identifiant ou le mot de passe qui est faux. On ne donne pas d'indices à l'ennemi.

5. Mission 4 — Dossiers classifiés (autorisation)

[20 pts]

Tous les agents ne sont pas égaux. Mettez en place l'**autorisation par rôle**.

Objectif opérationnel

- Les routes de **consultation** des missions sont accessibles à **tout agent authentifié**.
- **Seul le CHEF** peut **créer**, **assigner** et **supprimer** une mission. Un agent ordinaire qui tente ces actions doit être **poliment éconduit** (refus d'autorisation).
- Distinguez clairement, dans vos réponses, « *je ne sais pas qui tu es* » et « *je sais qui tu es, mais tu n'as pas le droit* ».
- **Défi optionnel d'habilitation** : un agent ne peut consulter que les missions dont le niveau de confidentialité est *inférieur ou égal* à son habilitation.

Pensez à l'ordre des middlewares : on identifie l'agent *avant* de vérifier ses droits. Vous aurez besoin d'au moins un compte CHEF pour tester (à vous de trouver comment en obtenir un).

6. Mission 5 — Opération de validation

[10 pts]**Démonstration**

- Préparez un jeu de requêtes (fichier de tests REST, collection Postman, ou script) qui démontre **tout le flux** : recruter → s'inscrire → se connecter → accéder à une route protégée → se faire refuser une action réservée au CHEF.
- Montrez les **bons codes de statut** pour chaque cas (succès, non authentifié, non autorisé, introuvable).
- Prouvez la **persistance** : les données survivent à un redémarrage du serveur.

Objectifs bonus (pour les agents d'élite)

Facultatifs, mais ils impressionnent le QG :

- **Filtrage & pagination** des missions (par statut, par habilitation) via la *query string*.
- **Statistiques** : nombre de missions par statut, ou classement des agents par récompenses cumulées.
- **Validation** robuste des entrées (rejeter proprement les données malformées).
- **Expiration courte** du token + une route qui indique « profil de l'agent connecté ».
- **Journal de bord** (logger) qui trace chaque requête sensible.

Barème (sur 100) — conversion en +5 %

Critère	Points
Mission 0 — Projet, Git, connexion Neon	5
Mission 1 — Schéma Prisma (enums, relation, migration)	20
Mission 2 — Intégration Axios + persistance	20
Mission 3 — Authentification (hachage, token, middleware)	25
Mission 4 — Autorisation par rôle (401 vs 403)	20
Mission 5 — Démonstration et persistance	10
Total	100
Objectifs bonus (au mérite, plafonné)	+10

Rappel récompense : la note de ce laboratoire bonus est convertie en un gain de +5 % ajouté à l'un de vos 3 laboratoires (vous choisissez lequel). Un travail à 100/100 vaut donc l'intégralité du bonus.

Livrables

- Le **lien du dépôt Git** (sans le fichier de secrets, évidemment).
- Un court **README** : comment lancer le projet, et la liste des routes.
- Le **jeu de requêtes** de démonstration (Mission 5).
- Les **noms des agents** de la cellule.

ANNEXE — Aide-mémoire de terrain (rappels)

Cette annexe ne contient que des rappels de commandes et de concepts. Aucune solution n'y figure.

A.1 — Installation des dépendances

```
# Outils de base
npm install express dotenv
npm install --save-dev typescript tsx @types/node @types/express

# Base de données
npm install @prisma/client @prisma/adapters-neon
npm install --save-dev prisma

# Appels d'API externes
npm install axios

# Sécurité
npm install bcryptjs jsonwebtoken
npm install --save-dev @types/bcryptjs @types/jsonwebtoken
```

A.2 — Commandes Prisma utiles

```
npx prisma init           # initialiser Prisma
npx prisma migrate dev    # créer/appliquer une migration + générer le client
npx prisma generate       # régénérer le client après modification du schéma
npx prisma studio         # interface graphique pour voir/éditer les données
npx prisma migrate status # état des migrations
```

A.3 — Variables d'environnement (fichier .env)

```
DATABASE_URL="postgresql://...neon.tech/...?sslmode=require"
JWT_SECRET="une_longue_chaine_alatoire_secrete"
PORT=3000
```

Pour générer un secret aléatoire :

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

Ne committez jamais le fichier .env (ajoutez-le au .gitignore, avec node_modules/ et generated/).

A.4 — API publique suggérée (pour la Mission 2)

Random User Generator — gratuit, sans clé : <https://randomuser.me/api/>
Renvoie une fausse identité (nom, photo, nationalité, email, identifiant unique...) idéale comme « couverture ». Vous êtes libres d'en choisir une autre, du moment qu'elle est publique et sans authentification.

A.5 — Aide-mémoire des verbes HTTP

Verbe	Sens
GET	Lire
POST	Créer
PUT	Remplacer (tout)
PATCH	Modifier (en partie)
DELETE	Supprimer

A.6 — Aide-mémoire des codes de statut

Code	Quand l'utiliser
200	Succès (lecture, mise à jour)
201	Ressource créée
400	Requête invalide (données manquantes/malformées)
401	Non authentifié (token absent ou invalide)
403	Non autorisé (authentifié, mais pas le bon rôle)
404	Ressource introuvable
500	Erreur serveur

A.7 — Rappels de concepts

- **Authentification** (« qui es-tu ? ») vient *avant* l'**autorisation** (« as-tu le droit ? »).
- Un mot de passe se **hache** (sens unique), il ne se chiffre pas et ne se stocke jamais en clair.
- Un **JWT** est *signé*, pas *chiffré* : son contenu est lisible → n'y mettez aucun secret.
- Le client envoie son token dans l'en-tête **Authorization: Bearer <token>**.
- Convention REST : noms de ressources au *pluriel*, pas de verbe dans l'URL.
- HTTP est *sans état* : c'est le token (ou la session) qui « prouve » qu'on est connecté à chaque requête.

A.8 — Aide-mémoire des librairies (signatures)

Rappels de signatures publiques, pour vous orienter — l'implémentation reste à votre charge.

```

bcrypt.hash(motDePasse, 10)           // -> Promise<hash>
bcrypt.compare(motDePasse, hash)       // -> Promise<boolean>

jwt.sign(payload, secret, options)     // options : { expiresIn: '1h' }
jwt.verify(token, secret)              // lance une erreur si invalide/expire

axios.get(url, config)                 // config : { params, headers, timeout }
axios.post(url, corps, config)

prisma.<modele>.create({ data })
prisma.<modele>.findMany({ where, include, orderBy })
prisma.<modele>.findUnique({ where })
prisma.<modele>.update({ where, data })
prisma.<modele>.delete({ where })

```

FIN DE LA TRANSMISSION • Bonne chance, agents. Le QG compte sur vous.